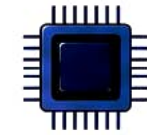
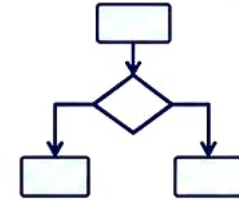


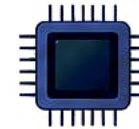
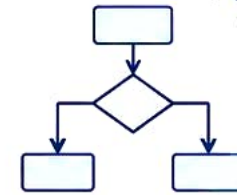
1010101010
0101010101
1010101010
0101010101



Notes

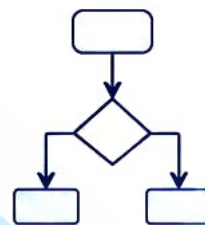


1010101010
0101010101
1010101010
0101010101



Unit-I

10110
01001
10101



COMPUTER NETWORKS

INTRODUCTION

- A computer network is a collection of two or more computers/devices connected together through communication links for the purpose of sharing resources and exchanging data.
- These devices can be computers, servers, printers, mobile phones or any other hardware capable of communication.
- The main aim of computer networks is to enable communication and resource sharing among users, no matter where they are located.
- A computer network may be small, such as a network connecting computers in a home, or large, such as the Internet, which connects millions of devices all over the world.

In simple words, Computer Network is the connection of computers/devices that allows them to communicate and share data and resources.

NEED OF COMPUTER NETWORKS

Computer networks are essential in today's world. Some of the major needs are:

- ① Resource Sharing : Allows sharing of hardware (printers, scanners, storage devices, etc.) and software among multiple users.
- ② Data Sharing : Enables easy transfer and sharing of data and information between users.
- ③ Communication : Makes communication fast and easy through email, chat, video conferencing and social media.
- ④ Centralized Data Management : Data can be stored at a central location, making it easy to manage, update and secure.
- ⑤ Cost Saving : Sharing resources like printers, internet connection and storage reduces overall cost.
- ⑥ Improved Efficiency and Productivity : Users can access needed information quickly, leading to better efficiency and productivity.
- ⑦ Remote Access : Allows users to access data and resources from anywhere in the world.

COMMUNICATION PROTOCOL

INTRODUCTION

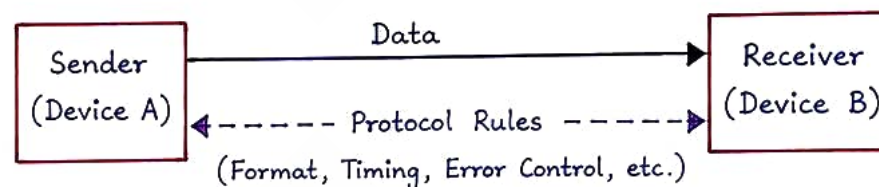
A Communication Protocol is a set of rules and standards that govern how data is transmitted, received and interpreted between devices in a network.

- It defines the format, timing, sequencing and error handling of data.
- It ensures that data sent by one device can be correctly understood by another device.
- Communication protocols enable effective and reliable communication between devices regardless of their hardware or software differences.

KEY FUNCTIONS

- Defines how data is formatted and transmitted.
- Manages the transmission and synchronization of data.
- Detects and corrects errors.
- Controls the flow of data between devices.
- Establishes, maintains and terminates communication sessions.

HOW COMMUNICATION PROTOCOL WORKS ?



- ① Sender creates data and sends it according to the protocol rules.
- ② Data is transmitted through the communication medium.
- ③ Receiver receives the data and checks it according to the protocol rules.
- ④ If any error is found, the protocol handles it (retransmission or correction).
- ⑤ Data is delivered correctly to the receiver.

EXAMPLES OF COMMUNICATION PROTOCOLS

- HTTP (Hyper Text Transfer Protocol) – Used for web communication.
- FTP (File Transfer Protocol) – Used for file transfer.
- TCP (Transmission Control Protocol) – Ensures reliable data delivery.
- IP (Internet Protocol) – Handles addressing and routing of data.
- SMTP (Simple Mail Transfer Protocol) – Used for sending emails.

* In simple words, Communication Protocol is like a language and rules that allow devices to communicate, understand and exchange information effectively. *

LAN (LOCAL AREA NETWORK)

INTRODUCTION

A Local Area Network (LAN) is a network that connects computers and devices within a limited area such as a room, building, office or campus.

KEY FEATURES OF LAN

- Covers a small geographical area.
- High data transfer rate (usually 100 Mbps to 10 Gbps or more).
- Owned and managed by a single organization.
- Provides easy resource sharing (files, printers, internet, applications, etc.).
- More secure and reliable.
- Low installation and maintenance cost compared to WAN.

EXAMPLES OF LAN

- Network in a home
- Network in a school computer lab
- Network in an office
- Network in a college department

COMPONENTS OF LAN

1. Network Devices – Computers, laptops, printers, servers, etc.
2. Network Interface Card (NIC) – Installed in each device to connect to the network.
3. Transmission Media – Cable (UTP, Coaxial, Fiber Optic) or Wireless (Wi-Fi).
4. Networking Devices – Hub, Switch, Router, Access Point, etc.
5. Network Software – Operating System, network protocols, drivers, etc.

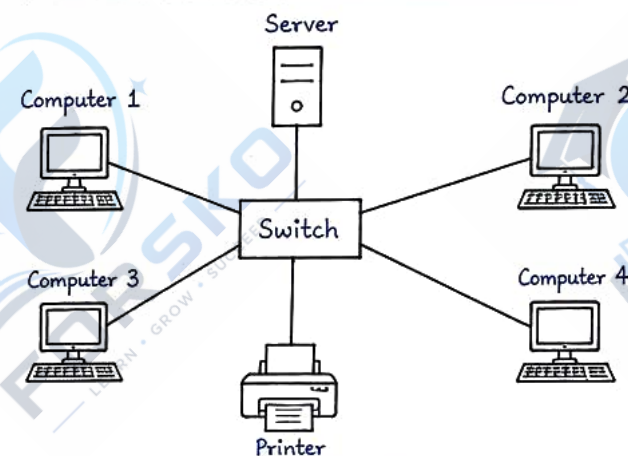
ADVANTAGES OF LAN

- Easy communication and data sharing.
- Sharing of hardware like printers and storage devices.
- High speed and efficiency.
- Centralized management and control.
- Cost-effective for limited area.
- Improves productivity and collaboration.

TYPES OF LAN (BASED ON TOPOLOGY)

- Bus Topology
- Star Topology
- Ring Topology
- Mesh Topology
- Tree Topology

LAN TOPOLOGY – STAR (MOST COMMON)



APPLICATIONS OF LAN

- File and data sharing
- Internet access sharing
- Sharing printers and other devices
- Communication (Email, Chat, VoIP)
- Used in offices, schools, hospitals, libraries, etc.

In simple words, LAN is a network that connects devices within a small area to share resources, exchange data and communicate with each other efficiently. 😊

MAN (METROPOLITAN AREA NETWORK)

INTRODUCTION

A Metropolitan Area Network (MAN) is a network that covers a large geographical area, such as a city or a metropolitan region. It is larger than a LAN but smaller than a WAN.

MAN is designed to connect multiple LANs within a city or campus.

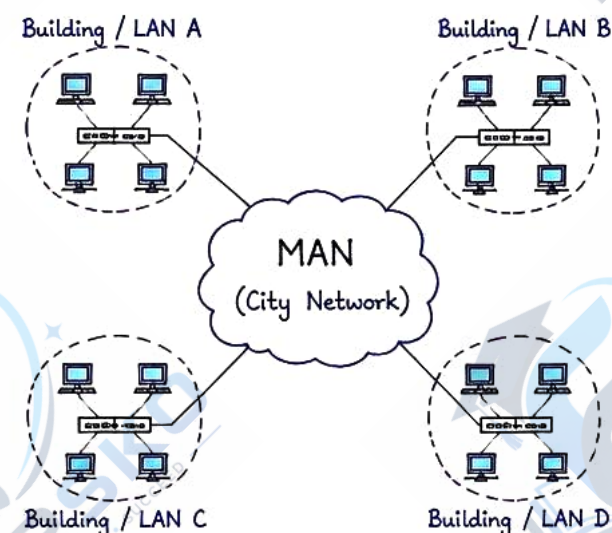
KEY FEATURES OF MAN

- Covers a city or metropolitan area.
- Connects multiple LANs and buildings.
- Provides high speed and reliable connectivity.
- Supports large amount of data and voice communication.
- Managed by organizations, ISPs or telecommunication companies.
- Example technologies : Metro Ethernet, Fiber Optic, Wi-Fi, Microwave.

EXAMPLES OF MAN

- Cable TV network in a city.
- Bank branches connected within a city.
- University or college network across different campuses in a city.
- Government organizations network.

MAN TOPOLOGY (EXAMPLE)



ADVANTAGES OF MAN

- High speed data transfer.
- Connects multiple LANs across the city.
- Cost effective than WAN.
- Improves communication and collaboration between organizations.
- Centralized data and resource sharing.
- Reliable and secure network.

DISADVANTAGES OF MAN

- Cost of installation is high.
- Requires skilled professionals to manage.
- Dependent on service providers.
- Complex to design and maintain.

MAN vs LAN vs WAN

Feature	LAN	MAN	WAN
Full Form	Local Area Network	Metropolitan Area Network	Wide Area Network
Coverage Area	Room, Building, Campus	City or Metropolitan Area	Country, Continent or Global
Speed	Very High	High	Lower than LAN/MAN
Ownership	Private	Private / ISP / Organization	ISP / Telecom Providers
Example	Office LAN	City Network	Internet

APPLICATIONS OF MAN

- Interconnection of banks in a city.
- University networks between different campuses.
- Government offices communication.
- Business organizations across the city.
- Public safety and emergency services network.

In simple words, MAN is a network that connects multiple LANs in a city or metropolitan area to share resources and communicate efficiently. 😊

WAN (WIDE AREA NETWORK)

INTRODUCTION

A Wide Area Network (WAN) is a network that spans a large geographical area such as a country, continent or even the whole world. It connects multiple LANs, MANs or individual devices over long distances using public or private communication links.

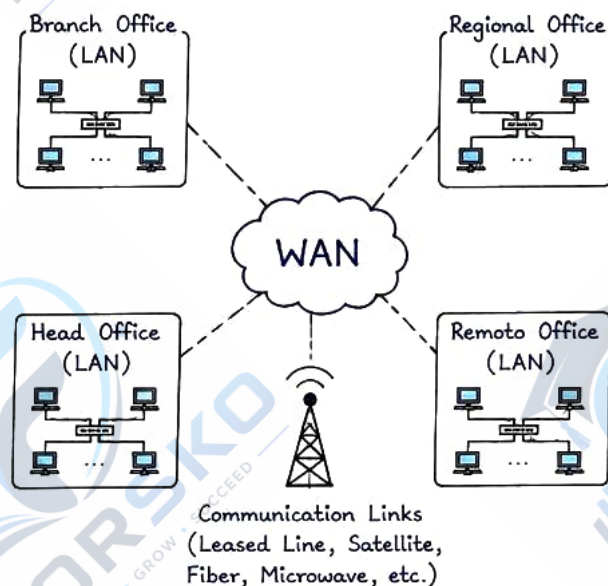
KEY FEATURES OF WAN

- Covers very large geographical areas.
- Connects multiple LANs/MANs.
- Uses public or private communication channels.
- Supports high speed and reliable data transfer over long distances.
- More expensive to install and maintain compared to LAN or MAN.
- Managed by telecom service providers or large organizations.

EXAMPLES OF WAN

- The Internet (largest WAN in the world).
- Bank networks connecting branches in different cities.
- Airline reservation systems.
- Corporate networks connecting offices in different countries.
- Government networks.

WAN TOPOLOGY (EXAMPLE)



WAN vs MAN vs LAN

Feature	LAN	MAN	WAN
Full Form	Local Area Network	Metropolitan Area Network	Wide Area Network
Coverage Area	Room, Building, Campus	City or Metropolitan Area	Country, Continent or Global
Speed	Very High	High	Lower than LAN and MAN
Cost	Low	Moderate	High
Ownership	Private	Private / ISP	ISP / Telecom Providers
Example	Office LAN	City Network	Internet

HOW WAN WORKS

- Data from a device in one location is sent to a local network (LAN).
- The local network forwards the data to the WAN through a router or modem.
- Data travels across long-distance communication links.
- It reaches the destination network, where it is delivered to the required device.

ADVANTAGES OF WAN

- Connects organizations across cities, countries and the world.
- Enables communication and data sharing globally.
- Supports centralized data and resource sharing.
- Improves business operations and collaboration.
- Allows access to global services like Internet.

DISADVANTAGES OF WAN

- High installation and operating cost.
- Lower speed compared to LAN.
- More complex to design and manage.
- Dependent on service providers.
- Security risks over public networks.

APPLICATIONS OF WAN

- Connecting company branches in different cities.
- International business communication.
- E-mail, VoIP and video conferencing.
- Online banking and financial transactions.
- Access to cloud services and web applications.

In simple words, WAN is a network that connects multiple local networks over large distances to share data and resources across the world. 😊

RING TOPOLOGY

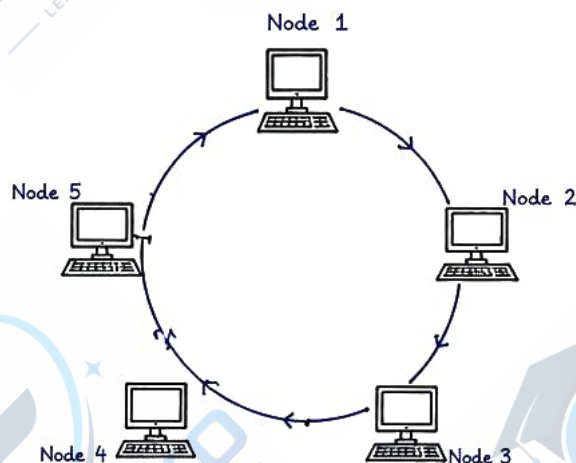
INTRODUCTION

In a Ring Topology, each device is connected to exactly two other devices, forming a circular network. Data travels in one direction (or both directions) around the ring until it reaches the destination.

KEY FEATURES

- Each device has two neighbors.
- Data flows in a circular path.
- Usually uses a special token to control access to the network.
- All devices have equal access to the network.
- Easy to add or remove devices without disturbing the network.

DIAGRAM (EXAMPLE)



HOW IT WORKS

When a device wants to send data, it waits for a token (permission). After getting the token, it sends the data to the next device in the ring. The data keeps moving from one device to another until it reaches the destination. The destination copies the data and releases the token for others.

ADVANTAGES

- Easy to install and expand.
- No central device is required.
- Equal access for all devices.
- Performs well under heavy network load.
- Fault in one link does not affect the whole network (if dual ring is used).

DISADVANTAGES

- If one device fails, it can break the whole ring (in single ring).
- Troubleshooting can be difficult.
- Performance depends on the number of devices.
- Adding or removing devices may disturb the network.

APPLICATIONS

- Used in small offices and local networks.
- Used in Token Ring networks.
- Suitable for environments with moderate traffic.

RING TOPOLOGY - SUMMARY

Feature	Description
Structure	Devices are connected in a circular (ring) form.
Data Flow	Flows in one direction (or both directions) around the ring.
Fault Tolerance	Low in single ring, High in dual ring.
Cost	Low to moderate.
Performance	Good, but depends on the number of nodes.

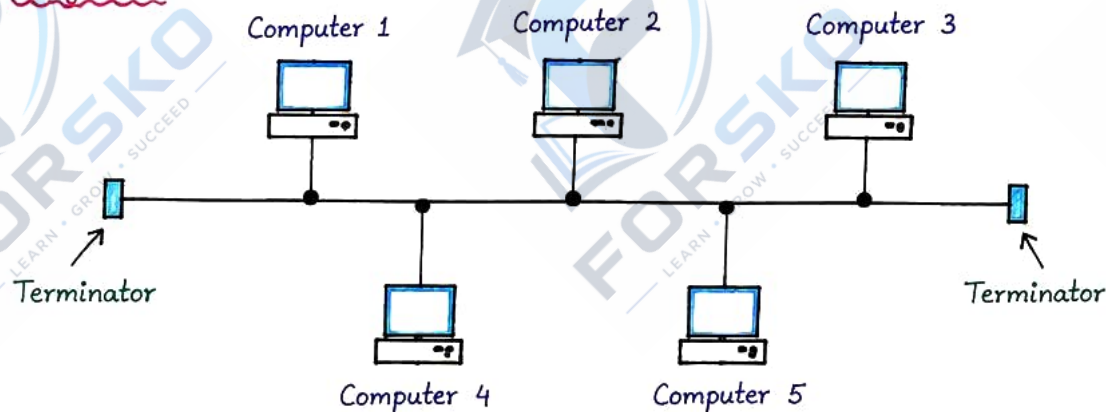
In simple words, Ring Topology connects each device to two others in a circle. Data travels around the ring until it reaches the correct device. 😊

Bus Topology

Definition :

In Bus Topology, all the devices are connected to a single main cable called Backbone Cable (Bus). Data travels in both directions along the bus and each device receives the data, but only the intended device accepts it.

Diagram :



Working :

- When a device wants to send data, it places the data on the bus.
- The data travels in both directions.
- All devices receive the data but only the destination device accepts it.
- Terminators at both ends absorb the signal and prevent reflections.

Characteristics :

- Uses a single backbone cable.
- All devices are connected to this backbone cable.
- Easy to install and extend.
- Less cable is required as compared to other topologies.
- If the backbone cable fails, the whole network fails.

Advantages :

- Easy to install.
- Requires less cable.
- Cost effective.
- Easy to extend the network.

Disadvantages :

- If backbone cable fails, entire network fails.
- Difficult to detect faults.
- Performance decreases with more devices.
- Limited length of backbone cable.

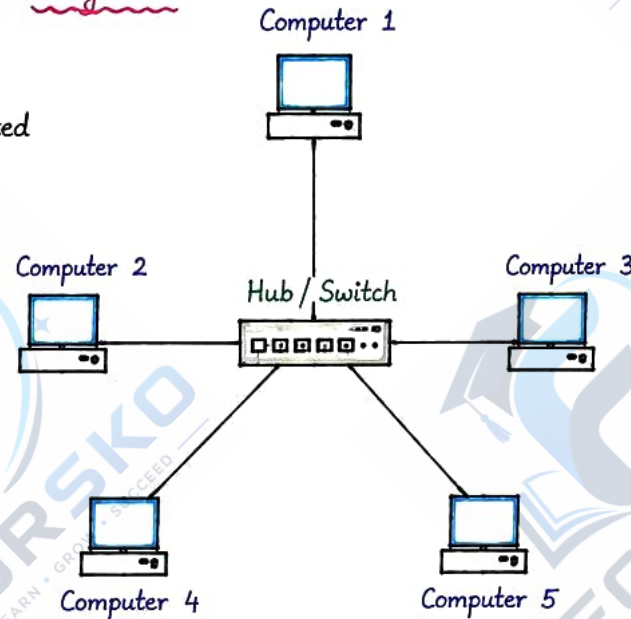
Example : Used in small offices, labs, temporary networks.

★ Star Topology ★

Definition :

In Star Topology, all the devices are connected to a central device called Hub or Switch. All the data is transmitted through this central device. Each device has a dedicated point-to-point connection with the central device.

Diagram :



Characteristics :

- All devices are connected to a central hub or switch.
- Each device has its own dedicated connection.
- Data is transmitted through the central device.
- Easy to install and manage.
- Failure of one device does not affect others.

Advantages :

- Easy to install and troubleshoot.
- Easy to add or remove devices.
- Failure of one device does not affect the whole network.
- Better performance with a switch.

Disadvantages :

- If the central hub/switch fails, the whole network stops.
- Requires more cable than bus topology.
- Cost is higher than bus topology.

Working :

- When a device wants to send data, it sends the data to the central hub/switch.
- The hub/switch receives the data and forwards it to the intended device.
- All communication between devices must pass through the central hub/switch.

Example :

Used in offices, computer labs, schools, and home networks.

Note : Star Topology is the most commonly used topology in modern networks because it is reliable, easy to manage and scalable.

Mesh Topology

Definition :

In Mesh Topology, every device is connected to every other device in the network. There is a dedicated point-to-point link between every pair of devices.

Characteristics :

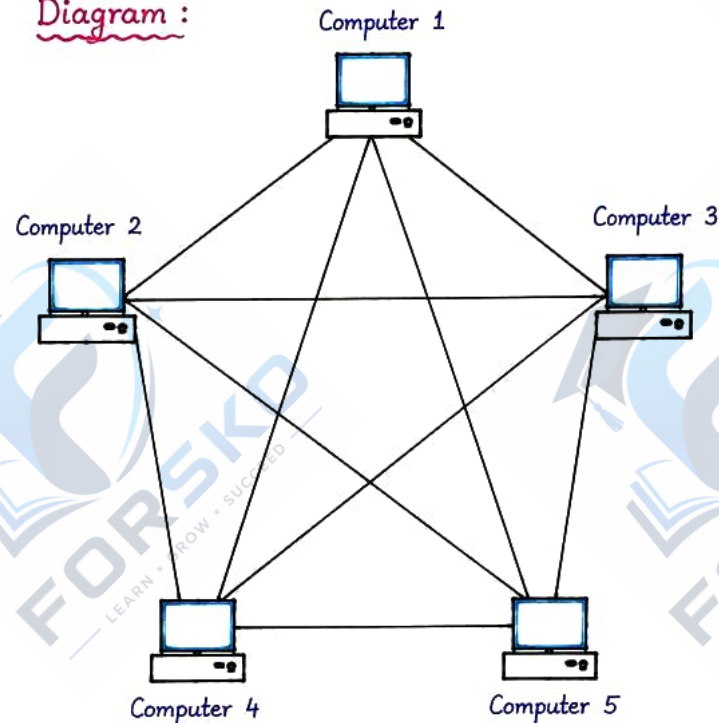
- Every device has a dedicated link to every other device.
- Data can take multiple paths to reach the destination.
- Very reliable and fault tolerant.
- Expensive and complex cabling.
- Difficult to install and manage.

Working :

- When a device wants to send data to another device, it can send the data through any of the available multiple paths.
- Since every device is connected to every other device, the network remains active even if one link fails.

Note : Mesh Topology is the most reliable topology but it is expensive and complicated to install. It is mostly used in situations where reliability is more important than cost.

Diagram :



Advantages :

- Very reliable.
- If one link fails, data can still be transmitted through other links.
- High security and privacy.
- Fault isolation is easy.
- Provides multiple paths for data transmission.

Disadvantages :

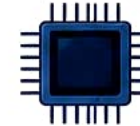
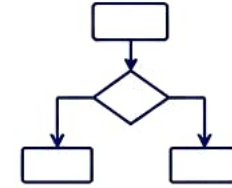
- Requires large amount of cable.
- Installation is difficult and costly.
- More ports are required in each device.
- Not suitable for large networks due to high cost.

Example :

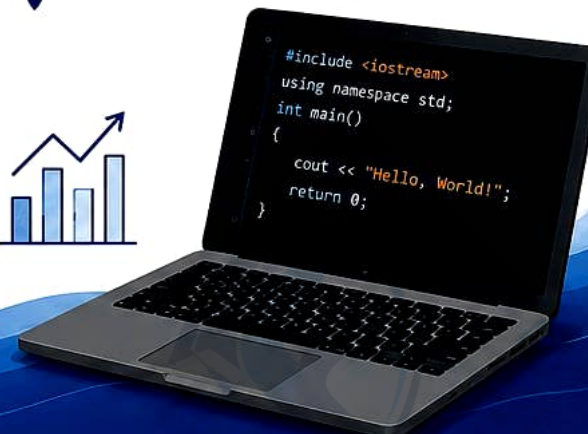
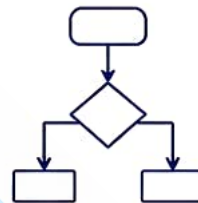
Used in military communication, banking networks, and critical applications where reliability is essential.

Unit - II

1010101010
0101010101
1010101010
0101010101



10110
01001
10101



NETWORK DEVICES: REPEATER

INTRODUCTION

A Repeater is a network device that receives a weak or degraded signal from one segment of a network and regenerates it to full strength before transmitting it to the next segment.

It works at the Physical Layer (Layer 1) of the OSI model.

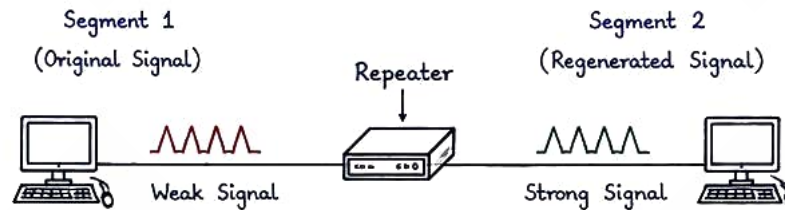
KEY FEATURES

- Regenerates and retimes the signal.
- Extends the length of a network.
- Does not change or understand data.
- Works on electrical, optical or wireless signals.
- Simple and easy to install.

WHERE REPEATERS ARE USED?

- In large networks to extend the maximum cable length.
- In environments where the signal gets weak due to distance or interference.
- In Ethernet networks (Hub based) and older network systems.
- In wireless communication to boost weak signals.

DIAGRAM



The repeater receives the weak signal from Segment 1, regenerates it and sends the strong signal to Segment 2.

EXAMPLE

Suppose two computers are far away from each other and the signal becomes weak in between. A repeater is placed in between them. It receives the weak signal, regenerates it and sends the strong signal further so that data can reach the destination correctly.



In simple words, A Repeater is like a booster. It takes a weak signal, cleans it, strengthens it and sends it forward. 😊

FUNCTIONING

1. Receives the incoming signal from the previous segment.
2. Amplifies and regenerates the signal to original strength.
3. Reshapes and retimes the signal to reduce noise and distortion.
4. Forwards the regenerated signal to the next segment.

IMPORTANT POINT

- Repeater does not filter or correct errors. It only regenerates the signal.

ADVANTAGES

- Extends network distance.
- Improves signal quality.
- Simple and cost effective.
- Easy to install and maintain.

DISADVANTAGES

- Does not filter or correct errors.
- Cannot reduce network congestion.
- Not intelligent device (works only at Physical Layer).

NETWORK DEVICES: HUB

INTRODUCTION

A Hub is a basic networking device that connects multiple devices together in a network. It works at Physical Layer (Layer 1) of the OSI model. It receives data from one device and broadcasts it to all other connected devices.

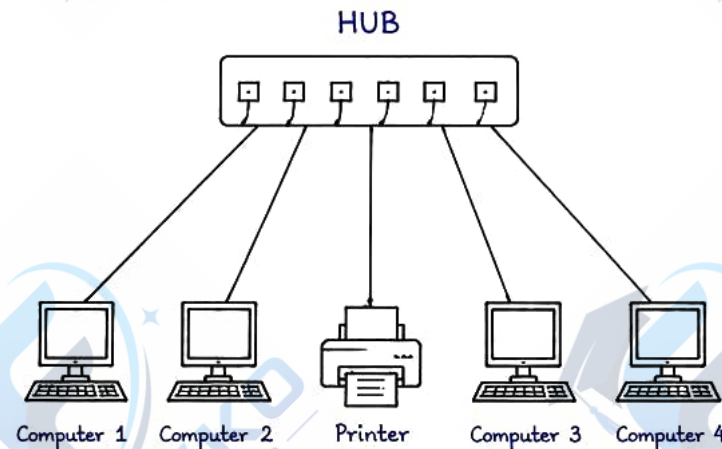
KEY FEATURES

- Operates at Physical Layer (Layer 1).
- Broadcasts incoming data to all ports.
- No filtering or switching of data.
- All ports share the same bandwidth.
- Simple and easy to install.
- Used to connect multiple devices in a network.

WHERE HUBS ARE USED?

- In small networks where cost is the main factor.
- In networks where high speed and performance are not critical.
- To connect devices like computers, printers, IP cameras, etc.

DIAGRAM



When any device sends data to the hub, the hub copies the data and sends it to all other connected devices.

HOW HUB WORKS

1. A device sends data to the hub.
2. The hub receives the data.
3. The hub repeats (broadcasts) the data to all other ports.
4. All devices receive the data, but only the intended device accepts it.

IMPORTANT POINTS

- Hub does not know the destination address.
- It sends data to all devices, which may cause unnecessary traffic.
- Hub does not improve network performance.

TYPES OF HUB

- Passive Hub - Just connects devices.
- Active Hub - Regenerates and boosts the signal before sending it to other ports.

ADVANTAGES

- Very simple and easy to use.
- Low cost.
- Easy to install.
- Can connect many devices.

DISADVANTAGES

- All devices share the same bandwidth.
- More collisions, less performance.
- No security (data is visible to all devices).
- Not suitable for large networks.

In simple words, A Hub is like a common connection point in a network. Whatever data comes to it, it sends to all devices. 😊

NETWORK DEVICES : SWITCH

1. What is a Switch?

A Switch is a networking device that connects multiple devices within a LAN (Local Area Network) and forwards data to the intended device only.

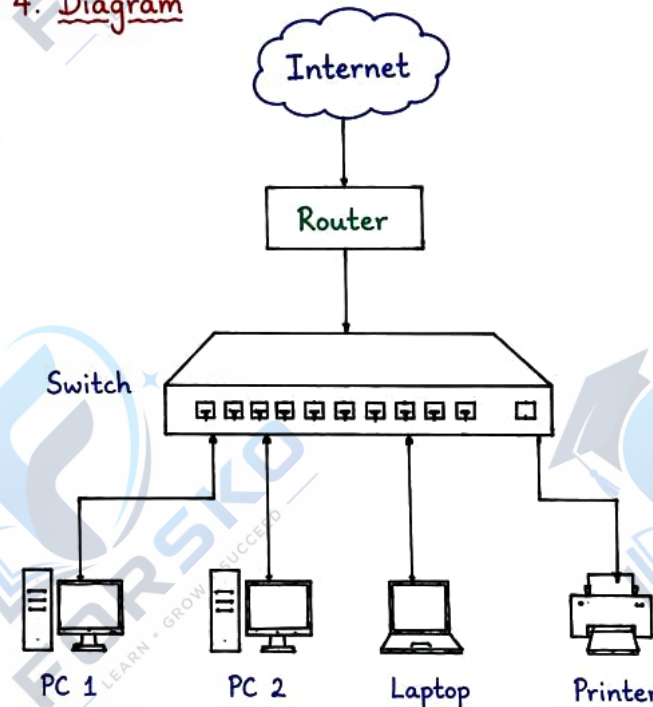
2. Key Features

- Operates at Data Link Layer (Layer 2) of OSI model.
- Uses MAC addresses to forward data.
- Reduces network congestion by sending data only to the destination port.
- Supports full-duplex communication.
- Plug and Play device.

3. Types of Switches

1. Unmanaged Switch – Basic, no configuration.
2. Managed Switch – Configurable and monitorable.
3. Smart Switch – Between unmanaged and managed switch.

4. Diagram



5. How Switch Works?

- When a device sends data, the switch reads the destination MAC address.
- It checks its MAC address table.
- If the address is found, it forwards the data only to that port.
- If not found, it floods the data to all ports except the incoming port.
- This process reduces unnecessary traffic.

6. MAC Address Table

Switch maintains a MAC address table to remember which device is connected to which port.

MAC Address	Port
00:11:22:33:44:55	1
00:11:22:33:44:66	2
00:11:22:33:44:77	3
...	...

This table is updated automatically.

7. Advantages

- ✓ Improves network performance.
- ✓ Reduces collision and congestion.
- ✓ Secure data transmission.
- ✓ Easy to install and use.

Summary

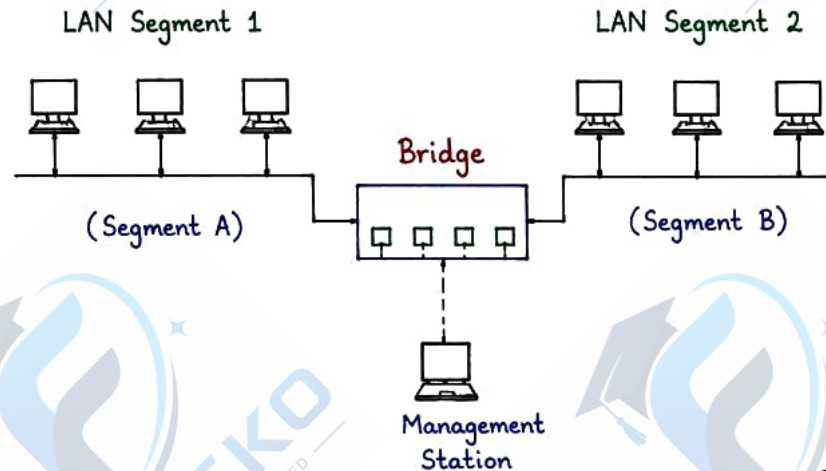
A switch is an intelligent device that connects devices in a LAN and ensures data is sent only to the right destination using MAC addresses.

NETWORK DEVICE : BRIDGE

1. What is Bridge?

A Bridge is a networking device that connects two or more LAN segments and forwards data based on MAC addresses. It operates at the Data Link Layer (Layer 2) of the OSI model.

2. Diagram



3. Key Features

- Connects multiple LAN segments.
- Operates at OSI Layer 2 (Data Link Layer).
- Uses MAC addresses to filter and forward frames.
- Reduces traffic and network congestion.
- Supports different LAN technologies.

4. How Bridge Works?

- When a frame arrives on a port, the bridge reads the source MAC address and records it in its MAC address table.
- It checks the destination MAC address in the table.
- If the destination is on the same segment, the frame is filtered (not forwarded).
- If the destination is on a different segment, the frame is forwarded.
- If the destination is unknown, the frame is flooded to all other ports except the incoming port.

5. MAC Address Table

MAC Address	Port	Time
00:11:22:33:44:55	1	10:15:30
00:11:22:33:44:66	2	10:15:45
00:11:22:33:44:77	3	10:15:55
...	...	...

(Table is updated dynamically)

6. Advantages

- Reduces unnecessary traffic.
- Improves network performance.
- Extends the distance between network segments.
- Enhances overall network efficiency.

7. Limitations

- Bridges cannot connect different network protocols.
- More complex than a repeater.
- Limited number of ports.

8. Example

Connecting two Ethernet LANs located on different floors of a building to reduce traffic and improve performance.

NETWORK DEVICE : ROUTER

1. What is Router ?

A Router is a networking device that connects two or more different networks and forwards data packets between them. It operates at Network Layer (Layer 3) of the OSI model.

2. Key Features

- Connects multiple networks.
- Forwards data packets based on IP addresses.
- Uses routing table to determine the best path.
- Reduces broadcast traffic by creating separate collision domains.
- Supports different routing protocols.
- Provides security through features like ACL, Firewall, NAT.

6. Types of Routing

- Static Routing – Routes are manually configured by the administrator.
- Dynamic Routing – Routes are learned automatically using routing protocols.

Examples of Routing Protocols :
RIP, OSPF, EIGRP, BGP

9. Important Terms

IP Address : Logical address used for routing packets.

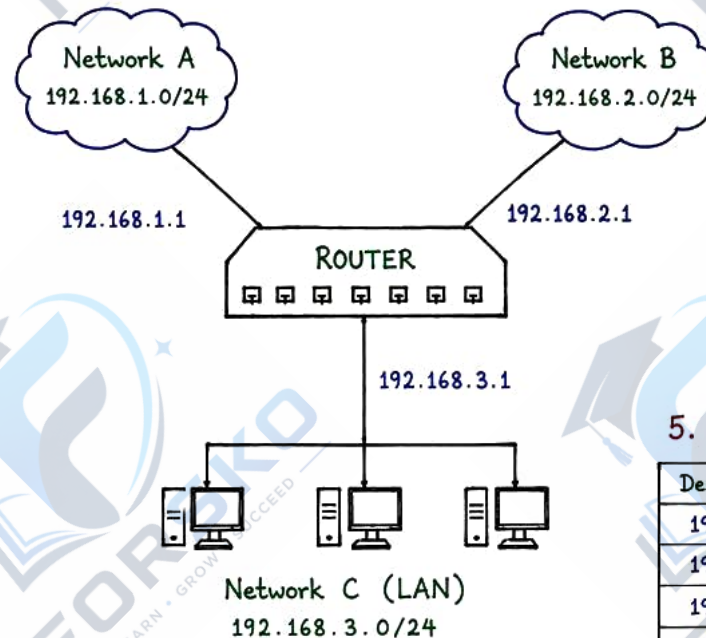
Routing Table : Table that stores routes to different networks.

Gateway : Router acts as gateway between different networks.

Next Hop : The next router or device to which the packet is sent.

Metric : A value used to determine the best route.

3. Diagram



4. How Router Works ?

- When a packet arrives, the router reads the destination IP address.
- It checks its routing table for the best path to that network.
- If a route is found, the packet is forwarded to the next hop.
- If no route is found, the packet is dropped and an error message may be sent.
- This process continues until the packet reaches the destination network.

5. Routing Table (Example)

Destination Network	Next Hop	Interface	Metric
192.168.1.0/24	Direct	192.168.1.1	0
192.168.2.0/24	Direct	192.168.2.1	0
192.168.3.0/24	Direct	192.168.3.1	0
0.0.0.0/0	192.168.2.254	192.168.2.1	1

0.0.0.0/0 is the default route (for unknown networks).

7. Advantages

- ✓ Connects multiple networks together.
- ✓ Finds the best path for data delivery.
- ✓ Improves performance by reducing congestion.
- ✓ Provides scalability for growing networks.
- ✓ Supports security and access control.

8. Common Uses

- ★ Connecting LAN to LAN.
- ★ Connecting LAN to the Internet.
- ★ Branch office connectivity.
- ★ Interconnecting different network technologies.
- ★ Used in homes, offices and ISPs.

OSI REFERENCE MODEL : INTRODUCTION

1. Introduction

The OSI (Open Systems Interconnection) Reference Model is a conceptual framework developed by ISO (International Standards Organization) to standardize the functions of a telecommunication or computer system into seven distinct layers.

It helps different systems and devices from different manufacturers to communicate with each other in a standard way.

2. Purpose

- Provides a common reference for network communication.
- Promotes interoperability between different systems.
- Breaks down a complex communication process into manageable layers.
- Each layer performs specific functions and services the layer above it.

6. Benefits of OSI Model

- ★ Makes network design and troubleshooting easier.
- ★ Allows development of modular and flexible networking systems.
- ★ Encourages vendor independence.
- ★ Facilitates learning and understanding of network operations.

3. The OSI Model - 7 Layers

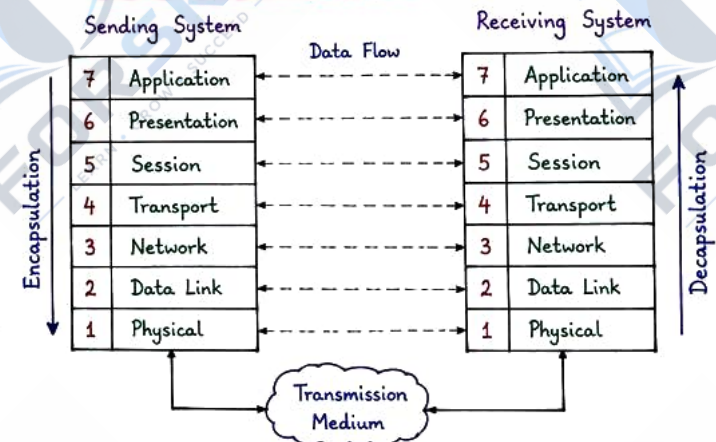
The OSI model consists of seven layers. Data flows from the top layer (Application Layer) down to the bottom layer (Physical Layer) when sending, and from bottom to top when receiving.

Layer No.	Layer Name	Function (In Short)
7	Application Layer	Provides services to user applications.
6	Presentation Layer	Handles data format, encryption, compression.
5	Session Layer	Manages sessions (start, manage, end) between applications.
4	Transport Layer	Ensures end-to-end delivery, error control, flow control.
3	Network Layer	Handles logical addressing and routing of packets.
2	Data Link Layer	Provides node-to-node delivery, framing, error detection.
1	Physical Layer	Transmits raw bits over the physical medium.

4. Key Features

- Layered Architecture - Divides the communication process into 7 layers.
- Standardization - Provides a standard for designing and implementing network protocols.
- Independent Layers - Changes in one layer do not affect other layers.
- Encapsulation - Data is wrapped with control information as it moves down the layers.
- Decapsulation - Control information is removed as data moves up the layers.

5. How OSI Model Works



7. In Summary

The OSI Reference Model is not a protocol but a model. It serves as a guide for understanding how data travels from one application to another across a network using seven layers, each with its own responsibilities.

Note :

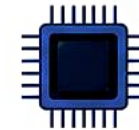
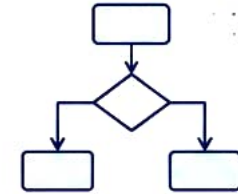
The OSI model is a theoretical model. The actual implementation in real-world networks may use a different number of layers or combine multiple layers.

FUNCTIONS OF SEVEN LAYERS OF OSI MODEL

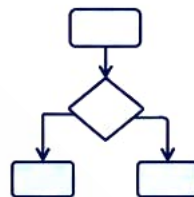
Layer	Layer Name	Functions
⑦	Application Layer	- Provides network services directly to the user applications. It allows user to access the network. - Examples : HTTP, FTP, SMTP, DNS, Telnet, SNMP etc.
⑥	Presentation Layer	- Handles data representation and translation. It performs encryption/decryption, compression/decompression and conversion of data formats. - Ensures that data is in a usable format for the application layer.
⑤	Session Layer	Establishes, manages and terminates sessions between applications. It provides dialog control (half/full duplex) and synchronization between communicating devices.
④	Transport Layer	- Provides end-to-end communication services. It performs segmentation and reassembly of data, flow control, error control and reliable data transfer. - Examples : TCP, UDP
③	Network Layer	- Handles logical addressing and routing of packets. It selects the best path for data delivery across multiple networks. - Example : IP (Internet Protocol)
②	Data Link Layer	- Provides node-to-node delivery of data. It handles framing, MAC addressing, error detection and correction, and flow control. - It ensures reliable data transfer over the physical link.
①	Physical Layer	- Transmits raw bits (0s and 1s) over the physical medium. It defines electrical, mechanical, procedural and functional specifications for transmission. - Examples : Cables, Hubs, Repeaters, Connectors, Wireless signals etc.

Unit - III

1010101010
0101010101
1010101010
0101010101



10110
01001
10101



* String Methods: Formatting Strings *

→ Formatting strings means inserting values into a string in a specified format.

→ Python provides different ways to format strings.

1) Using % operator (Old style formatting)

"string" % values

Example:

```
name = "Saqib"
age = 20
S = "My name is %s and I am %d years old." % (name, age)
print(s)
```

Output: My name is Saqib and I am 20 years old.

2) Using str.format() method (Recommended)

"string {}".format(values)

Example:

```
name = "Saqib"
age = 20
S = "My name is {} and I am {} years old.".format(name, age)
print(s)
```

Output: My name is Saqib and I am 20 years old.

More Examples:

- "{} {}".format("Hello", "World") → Hello World
- "{} is {} years old".format("Ali", 18) → Ali is 18 years old
- "{name} got {marks} marks".format(name="Sara", marks=95) → Sara got 95 marks
- f"Square of {5} is {5*5}" → Square of 5 is 25

3) Using f-strings (Python 3.6+) (Best & Easy)

f"string {variable}"

Example:

```
name = "Saqib"
age = 20
S = f"My name is {name} and I am {age} years old."
print(s)
```

Output: My name is Saqib and I am 20 years old.

Format Specifiers (with % or format())

Specifier	Meaning	Example
%s or {}	String	"%s" % "hello" → hello
%d or {}	Integer	"%d" % 10 → 10
%f or {}	Float	"%.2f" % 3.14159 → 3.14
%x or {}	Hexadecimal	"%x" % 255 → ff
%o or {}	Octal	"%o" % 8 → 10
%%	Percent sign	"%" % " → %

Key Points:

- % operator is old style.
- str.format() is more readable.
- f-strings are easiest and fastest.
- Formatting helps to create clean and professional output.

String Methods: Lists

- Lists are used to store multiple items in a single variable.
- Python provides a variety of built-in methods to work with Lists.

1) Creating a List

`[]` → empty List
`[item1, item2, item3, ...]` → List with items

Example:

```
fruits = ["apple", "banana", "mango"]
```

2) Common List Methods

No.	Method	Description	Example	Output
①	<code>append(x)</code>	Adds an item <code>x</code> to the end of the List	<code>Lst.append(10)</code>	<code>[1, 2, 3, 10]</code>
②	<code>insert(i, x)</code>	Inserts item <code>x</code> at index <code>i</code>	<code>Lst.insert(1, 99)</code>	<code>[1, 99, 2, 3]</code>
③	<code>extend(iterable)</code>	Adds all items from iterable to the List	<code>Lst.extend([4, 5])</code>	<code>[1, 2, 3, 4, 5]</code>
④	<code>remove(x)</code>	Removes first occurrence of <code>x</code>	<code>Lst.remove(2)</code>	<code>[1, 3]</code>
⑤	<code>pop([i])</code>	Removes and returns item at index <code>i</code> (or last item if <code>i</code> not given)	<code>Lst.pop()</code>	<code>[1, 2]</code> Returns 3
⑥	<code>index(x)</code>	Returns index of first occurrence of <code>x</code>	<code>Lst.index(2)</code>	1
⑦	<code>count(x)</code>	Returns number of occurrences of <code>x</code>	<code>Lst.count(2)</code>	2
⑧	<code>sort()</code>	Sorts the List in ascending order	<code>Lst.sort()</code>	<code>[1, 2, 3]</code>
⑨	<code>reverse()</code>	Reverses the order of items in the List	<code>Lst.reverse()</code>	<code>[3, 2, 1]</code>
⑩	<code>clear()</code>	Removes all items from the List	<code>Lst.clear()</code>	<code>[]</code>

3) Other Useful Methods

Method	Description	Example	Output
<code>copy()</code>	Returns a shallow copy of the List	<code>new = Lst.copy()</code>	<code>[1, 2, 3]</code>
<code>len(Lst)</code>	Returns number of items in the List	<code>len(Lst)</code>	3
<code>min(Lst)</code>	Returns the smallest item	<code>min([5, 2, 9])</code>	2
<code>max(Lst)</code>	Returns the largest item	<code>max([5, 2, 9])</code>	9
<code>sum(Lst)</code>	Returns the sum of all numeric items	<code>sum([1, 2, 3])</code>	6

4) Examples

① Basic Operations

```
Lst = [1, 2, 3]
Lst.append(4)      # [1, 2, 3, 4]
Lst.insert(1, 99)  # [1, 99, 2, 3, 4]
Lst.remove(2)      # [1, 99, 3, 4]
x = Lst.pop()      # x = 4, Lst = [1, 99, 3]
Lst.sort()         # [1, 3, 99]
Lst.reverse()      # [99, 3, 1]
```

② With Count and Index

```
Lst = [10, 20, 10, 30]
print(Lst.count(10)) # 2
print(Lst.index(20)) # 1
print(len(Lst))      # 4
```

Key Points:

- Lists are ordered, mutable and allow duplicate items.
- Indexing starts at 0.
- Many List methods modify the List in place.
- Use `help(List)` in Python to see all methods.

Syntax Reminder

List_name.method(arguments)
 Example: `Lst.append(10)`

* String Methods: Creating Lists *

- A List is a collection of items enclosed in square brackets [] and separated by commas.
- Lists are ordered, changeable (mutable) and can store different data types.

1) Creating an Empty List

```
my_list = [] # empty list
```

Example:

```
my_list = []
print(my_list) → Output: []
```

2) Creating a List with Items

```
my_list = [item1, item2, item3, ...]
```

Example:

```
fruits = ["apple", "banana", "mango"]
print(fruits) → Output: ['apple', 'banana', 'mango']
```

3) Creating a List Using list() Constructor

```
my_list = list(iterable)
```

Example:

```
numbers = list((1, 2, 3, 4)) # from tuple
chars = list("abcd") # from string
print(numbers) → Output: [1, 2, 3, 4]
print(chars) → Output: ['a', 'b', 'c', 'd']
```

4) Creating a List in Different Ways - Examples

No.	Method	Code Example	Output
①	Empty list	a = [] print(a)	[]
②	List with items	b = [10, 20, 30] print(b)	[10, 20, 30]
③	Using list() with tuple	c = list((5, 6, 7)) print(c)	[5, 6, 7]
④	Using list() with string	d = list("hello") print(d)	['h', 'e', 'l', 'l', 'o']
⑤	Using range()	e = list(range(1, 6)) print(e)	[1, 2, 3, 4, 5]

5) Creating a List Using Comprehension

```
my_list = [expression for item in iterable]
```

Example:

```
squares = [x*x for x in range(1, 6)] # squares of 1 to 5
print(squares) → Output: [1, 4, 9, 16, 25]
```

Key Points:

- Lists are created using square brackets [].
- Elements are separated by commas.
- Lists can hold items of any data type.
- Use list() to convert other iterable objects into lists.
- List comprehension provides a compact way to create lists.

* String Methods: Basic List Operations *

- ➔ After creating a list, we can perform various operations on it.
- ➔ Python provides many built-in operations (methods and operators) to work with lists.

Example :

```
lst = [10, 20, 30, 40]
```

(Here we will use this list for all examples)

Basic List Operations

No.	Operation	Description	Syntax / Example	Output
①	Accessing Elements	Access element using index	lst[0] # first element lst[2]	10 30
②	Negative Indexing	Access from the end of the list	lst[-1] # last element lst[-2]	40 30
③	Slicing	Get a part (slice) of the list	lst[1:3] # from index 1 to 2 lst[:3] # from start to index 2 lst[2:] # from index 2 to end	[20, 30] [10, 20, 30] [30, 40]
④	Change Element	Change the value of an element	lst[1] = 99	[10, 99, 30, 40]
⑤	Add Element (at end)	Add an element at the end	lst.append(50)	[10, 20, 30, 40, 50]
⑥	Add Element (at index)	Add an element at specific index	lst.insert(2, 25)	[10, 20, 25, 30, 40]
⑦	Extend List	Add all elements of another list	lst.extend([60, 70])	[10, 20, 30, 40, 60, 70]
⑧	Remove Element (by value)	Remove first occurrence of value	lst.remove(30)	[10, 20, 40]
⑨	Remove Element (by index)	Remove element at given index	lst.pop(2) # removes element at index 2	[10, 20, 40] (removed element: 30)
⑩	Delete List	Delete entire list	del lst	lst no longer exists
⑪	Length of List	Find number of elements	len(lst)	4
⑫	Count an Element	Count occurrences of an element	lst.count(20)	1
⑬	Index of Element	Find index of first occurrence	lst.index(40)	3
⑭	Reverse List	Reverse the list	lst.reverse()	[40, 30, 20, 10]
⑮	Sort List	Sort the list in ascending order	lst.sort()	[10, 20, 30, 40]
⑯	Clear List	Remove all items from list	lst.clear()	[]

Note :

append(), insert(), extend(), remove(), pop(), clear(), reverse(), sort() → modify the list.
count(), index(), len() → return information about the list.

Important Points :

- Lists are ordered, mutable and allow duplicates.
- Indexing in lists starts from 0.
- List operations modify the list in place (if applicable) and return the updated list.

More Examples :

- Using append() and insert()

```
a = [1, 2, 3]
a.append(4)
a.insert(1, 99)
print(a)
```

Output :

[1, 99, 2, 3, 4]

- Using extend()

```
a = [1, 2]
b = [3, 4, 5]
a.extend(b)
print(a)
```

Output :

[1, 2, 3, 4, 5]

- Using pop()

```
a = [10, 20, 30, 40]
x = a.pop() # removes last
y = a.pop(1) # removes index 1
print(a, x, y)
```

Output :

[10, 30] 40 20

- Using sort() and reverse()

```
a = [3, 1, 4, 2]
a.sort() # ascending
print(a)
a.reverse()
print(a)
```

Output :

[1, 2, 3, 4]

[4, 3, 2, 1]

* String Methods: Indexing and Slicing in Lists *

- Lists are ordered collections. Each item in a list has a unique position called index.
- In Python, indexing starts from 0.
- We can access single items using indexing and multiple items using slicing.

1) Positive and Negative Indexing

Example: `lst = ['a', 'b', 'c', 'd', 'e']`

Index (Positive)	0	1	2	3	4
	'a'	'b'	'c'	'd'	'e'
Index (Negative)	-5	-4	-3	-2	-1

Note: For a list of length n ,
negative index = positive index - n

2) Slicing in Lists

Slicing means extracting a part (sub-list) of a list.

Syntax: `lst[start : stop : step]`

- start → starting index (included)
- stop → ending index (excluded)
- step → interval/step (default is 1)

Common Slicing Examples:

Expression	Meaning	Result	Explanation
<code>lst[1:4]</code>	From index 1 to 3	<code>['b', 'c', 'd']</code>	Starts at 1 (included) and goes till 4 (excluded)
<code>lst[2:]</code>	From index 2 to end	<code>['c', 'd', 'e', 'f']</code>	No stop → till end
<code>lst[:4]</code>	From start to index 3	<code>['a', 'b', 'c', 'd']</code>	No start → from beginning
<code>lst[:]</code>	Complete list	<code>['a', 'b', 'c', 'd', 'e', 'f']</code>	All elements
<code>lst[::2]</code>	Every 2nd element	<code>['a', 'c', 'e']</code>	Step = 2
<code>lst[1:5:2]</code>	From 1 to 4, step 2	<code>['b', 'd']</code>	Takes every 2nd element in range
<code>lst[::-1]</code>	Reverse the list	<code>['f', 'e', 'd', 'c', 'b', 'a']</code>	Step = -1 reverses the list

Indexing in Lists

Indexing means accessing an element of a list using its position (index).

Example:

`lst = ["apple", "banana", "cherry", "date", "elderberry"]`

Positive Index	0	1	2	3	4
	'apple'	'banana'	'cherry'	'date'	'elderberry'
Negative Index	-5	-4	-3	-2	-1

Accessing Elements (Examples):

```
lst[0] → "apple"      # first element
lst[2] → "cherry"     # third element
lst[4] → "elderberry" # last element
lst[-1] → "elderberry" # last element (from end)
lst[-3] → "cherry"    # third last element
```

Example:

`lst = ['a', 'b', 'c', 'd', 'e', 'f']`

Index:	0	1	2	3	4	5
	'a'	'b'	'c'	'd'	'e'	'f'

Key Points:

- Indexing is used to access a single element.
- Positive index starts from 0.
- Negative index starts from -1 (last element).
- Slicing returns a new list (does not change original list).
- In slicing, the start index is included but the stop index is excluded.
- Step can be positive, negative or omitted.

Built-In Functions Used on Lists

<u>Function</u>	<u>Syntax</u>	<u>Description</u>	<u>Example</u>
1. <code>len()</code>	<code>len(list)</code>	Returns the number of items in the list.	<code>lst = [10, 20, 30]</code> <code>len(lst) → 3</code>
2. <code>max()</code>	<code>max(list)</code>	Returns the largest item in the list.	<code>lst = [10, 20, 30]</code> <code>max(lst) → 30</code>
3. <code>min()</code>	<code>min(list)</code>	Returns the smallest item in the list.	<code>lst = [10, 20, 30]</code> <code>min(lst) → 10</code>
4. <code>sum()</code>	<code>sum(list)</code>	Returns the sum of all items in the list.	<code>lst = [10, 20, 30]</code> <code>sum(lst) → 60</code>
5. <code>sorted()</code>	<code>sorted(list)</code>	Returns a new sorted list (ascending order).	<code>lst = [30, 10, 20]</code> <code>sorted(lst) → [10, 20, 30]</code>
6. <code>reversed()</code>	<code>reversed(list)</code>	Returns a reverse iterator of the list.	<code>lst = [10, 20, 30]</code> <code>list(reversed(lst)) → [30, 20, 10]</code>
7. <code>any()</code>	<code>any(list)</code>	Returns True if at least one item is True.	<code>lst = [0, False, 3, ""]</code> <code>any(lst) → True</code>
8. <code>all()</code>	<code>all(list)</code>	Returns True if all items are True.	<code>lst = [1, 2, True, 5]</code> <code>all(lst) → True</code>
9. <code>enumerate()</code>	<code>enumerate(list)</code>	Returns index and item as pairs.	<code>lst = ['a', 'b', 'c']</code> <code>list(enumerate(lst)) → [(0, 'a'), (1, 'b'), (2, 'c')]</code>
10. <code>zip()</code>	<code>zip(list1, list2, ...)</code>	Returns an iterator of tuples by combining lists.	<code>l1 = [1, 2, 3]</code> , <code>l2 = ['a', 'b', 'c']</code> <code>list(zip(l1, l2)) → [(1, 'a'), (2, 'b'), (3, 'c')]</code>

Note: Some functions like `len()`, `max()`, `min()`, `sum()`, `sorted()` return a value or new list, while others return iterators.

★ Commonly Used!

`len()`, `max()`, `min()`, `sum()`, `sorted()`,
`any()`, `all()`, `enumerate()`, `zip()`

Lists Methods

Lists methods are used to perform operations on lists.

Method	Syntax	Description	Example
① append()	list.append(x)	Adds an element at the end of the list.	lst = [10, 20] lst.append(30) # lst is now [10, 20, 30]
② extend()	list.extend(iterable)	Extends the list by adding elements from the iterable.	lst = [10, 20] lst.extend([30, 40]) # lst is now [10, 20, 30, 40]
③ insert()	list.insert(index, x)	Inserts an element 'x' at the specified index.	lst = [10, 30] lst.insert(1, 20) # lst is now [10, 20, 30]
④ remove()	list.remove(x)	Removes the first occurrence of element 'x'.	lst = [10, 20, 30, 20] lst.remove(20) # lst is now [10, 30, 20]
⑤ pop()	list.pop(index)	Removes and returns the element at the given index. (Default index is -1 i.e. last element)	lst = [10, 20, 30] x = lst.pop() # x = 30, lst is now [10, 20]
⑥ clear()	list.clear()	Removes all the elements from the list.	lst = [10, 20, 30] lst.clear() # lst is now []
⑦ index()	list.index(x)	Returns the index of the first occurrence of 'x'.	lst = [10, 20, 30, 20] i = lst.index(20) # i = 1
⑧ count()	list.count(x)	Returns the number of times 'x' occurs in the list.	lst = [10, 20, 20, 30, 20] c = lst.count(20) # c = 3
⑨ sort()	list.sort(reverse=False)	Sorts the list in ascending order. (Set reverse=True for descending order)	lst = [30, 10, 20] lst.sort() # lst is now [10, 20, 30]
⑩ reverse()	list.reverse()	Reverses the elements of the list in place.	lst = [10, 20, 30] lst.reverse() # lst is now [30, 20, 10]
⑪ copy()	list.copy()	Returns a shallow copy of the list.	lst = [10, 20, 30] new_lst = lst.copy() # new_lst is [10, 20, 30]

* **Note**: Indexing in lists starts from 0.
Most methods modify the original list (in-place) except copy() which returns a new list.

Commonly Used Methods

* append(), extend(), remove(), pop(), sort(), reverse(), index(), count()

The del Statement

The **del** statement is used to delete (remove) elements from a list or delete the entire list.

Syntax :

<code>del list [index]</code>	→ deletes the element at the specified index.
<code>del list [start : end]</code>	→ deletes elements from start index to end-1 index.
<code>del list</code>	→ deletes the entire list.

Example 2 : Delete multiple elements

```
numbers = [10, 20, 30, 40, 50]
del numbers[1:4] # deletes elements from
                 index 1 to 3 (20, 30, 40)

print(numbers)
```

Output : [10, 50]

Example 1 : Delete an element

```
numbers = [10, 20, 30, 40]
del numbers[1] # deletes element at index 1 (20)
print(numbers)
```

Output : [10, 30, 40]

Example 3 : Delete the entire list

```
numbers = [10, 20, 30]
del numbers # deletes the entire list
```

After this statement,
the list no longer exists.

Note :

If we try to print the list now, Python will show an error.

★ **In short :** The **del** statement is used to remove elements or the entire list from memory permanently.

Dictionaries: Creating Dictionary

- A dictionary in Python is a collection of **key-value pairs**. It is written inside curly braces `{ }`, with each key separated from its value by a colon `( : )`, and each pair separated by a comma `( , )`.

1. Creating an Empty Dictionary

Method 1

```
d1 = { }
```

using curly braces

Method 2

```
d2 = dict( )
```

using dict() constructor

Example :

```
d1 = { }
```

Output

```
d2 = dict( )
```

```
{ }
```

```
print(d1)
```

```
{ }
```

```
print(d2)
```

Points to Remember :

- Keys must be unique and immutable (like string, number, tuple).
- Values can be of any data type and can be duplicate.
- Dictionaries are unordered (before Python 3.7).
- Since Python 3.7, dictionaries maintain insertion order.

2. Creating a Dictionary with Values

We can create a dictionary by placing key-value pairs inside `{ }`.

Syntax :

```
dict_name = { key1 : value1 , key2 : value2 , ... }
```

Example :

```
student = { 'name' : 'Saqib' , 'age' : 17 , 'course' : 'BSc CS' }
```

```
print(student) # Output
```

student →

Key	'name'	'age'	'course'
Value	'Saqib'	17	'BSc CS'

3. Creating Dictionary using dict() Constructor

We can also create a dictionary using dict() with key-value pairs as arguments.

Example :

```
d = dict(name = 'Saqib' , age = 17 , course = 'BSc CS')
```

```
print(d) # Output
```

```
{ 'name' : 'Saqib' , 'age' : 17 , 'course' : 'BSc CS' }
```

Dictionaries: Accessing and Modifying key:value Pairs in Dictionaries

1. Accessing Values in a Dictionary

We can access the value of a specific key using the key inside square brackets `[]`.

Syntax :

```
dict_name[key]
```

returns the value associated with key

Example :

```
student = { 'name': 'Saquib', 'age': 17, 'course': 'BSc CS' }
print(student['name'])    # Output → Saquib
print(student['age'])     # Output → 17
print(student['course'])  # Output → BSc CS
```

2. Accessing Values using get() Method

The `get()` method returns the value for the specified key. If the key does not exist, it returns `None` or a default value.

Syntax :

```
dict_name.get(key)
```

returns value if key exists
else returns `None`

Example :

```
print(student.get('age'))    # Output → 17
print(student.get('city'))   # Output → None
print(student.get('city', 'Not Found')) # Output → Not Found
```

3. Modifying (Updating) an Existing Value

We can update the value of an existing key by assigning a new value to that key.

Syntax :

```
dict_name[key] = new_value
```

Example :

```
student = { 'name': 'Saquib', 'age': 17, 'course': 'BSc CS' }
student['age'] = 18    # updating age
print(student)
```

Output →

Key	Value
'name'	'Saquib'
'age'	18
'course'	'BSc CS'

4. Adding a New key:value Pair

We can add a new key:value pair by assigning a value to a key that does not exist in the dictionary.

Example :

```
student['city'] = 'Akola'    # adding new key:value
print(student)
```

Output →

Key	Value
'name'	'Saquib'
'age'	18
'course'	'BSc CS'
'city'	'Akola'

Points to Remember :

- Use `[]` to access or update values.
- Use `get()` to safely access values.
- Updating an existing key changes its value.
- Adding a new key increases the size of the dictionary.

Dictionaries : Built-In Functions Used on Dictionaries

Python provides several built-in functions and methods that make working with dictionaries easy and efficient.

Function/Method	Description	Syntax	Example	Output
1. <code>len()</code>	Returns the number of key:value pairs in the dictionary.	<code>len(dict)</code>	<code>d = {'a':1, 'b':2, 'c':3}</code> <code>len(d)</code>	3
2. <code>dict.keys()</code>	Returns a view object containing all the keys in the dictionary.	<code>dict.keys()</code>	<code>d.keys()</code>	<code>dict_keys(['a', 'b', 'c'])</code>
3. <code>dict.values()</code>	Returns a view object containing all the values in the dictionary.	<code>dict.values()</code>	<code>d.values()</code>	<code>dict_values([1, 2, 3])</code>
4. <code>dict.items()</code>	Returns a view object containing all key:value pairs as tuples.	<code>dict.items()</code>	<code>d.items()</code>	<code>dict_items([('a', 1), ('b', 2), ('c', 3)])</code>
5. <code>dict.get(key, default)</code>	Returns the value for the specified key. If key is not found, returns default value (None if not given).	<code>dict.get(key, default)</code>	<code>d.get('b')</code> <code>d.get('x', 'Not Found')</code>	2 'Not Found'
6. <code>dict.update(other_dict)</code>	Updates the dictionary with key:value pairs from another dictionary.	<code>dict.update(other_dict)</code>	<code>d = {'a':1}</code> <code>d.update({'b':2, 'c':3})</code> <code>d</code>	<code>{'a':1, 'b':2, 'c':3}</code>
7. <code>dict.pop(key, default)</code>	Removes the specified key and returns its value. If key not found, returns default (or raises error if default not given).	<code>dict.pop(key, default)</code>	<code>d = {'a':1, 'b':2}</code> <code>d.pop('a')</code> <code>d.pop('x', 'Not Found')</code> <code>d</code>	1 'Not Found' <code>{'b':2}</code>
8. <code>dict.clear()</code>	Removes all items from the dictionary.	<code>dict.clear()</code>	<code>d = {'a':1, 'b':2}</code> <code>d.clear()</code> <code>d</code>	<code>{}</code>
9. <code>dict.copy()</code>	Returns a shallow copy of the dictionary.	<code>dict.copy()</code>	<code>d2 = d.copy()</code>	Copy of d

Quick Example (Using Most Functions)

```
d = {'name': 'Saqib', 'age': 17, 'city': 'Akola'}
print(len(d))           # 3
print(d.keys())          # dict_keys(['name', 'age', 'city'])
print(d.values())        # dict_values([17, 'Akola'])
print(d.items())         # dict_items([('name', 'Saqib'), ('age', 17), ('city', 'Akola')])
print(d.get('age'))      # 17
print(d.get('x', 'Not Found')) # Not Found
```

Methods vs Functions

- `len()` → function
- `keys()`, `values()`, `items()`, `get()`, `pop()`, `update()`, `clear()`, `copy()` → methods

(They are called using dot (.) on dictionary object.)

Points to Remember :

- Dictionaries are mutable, so their content can be changed using these methods.
- `keys()`, `values()`, `items()` return view objects which reflect changes in the dictionary.
- `get()` is safer than direct access using `[]` because it avoids `KeyError`.
- `pop()` removes the item permanently.

Dictionaries : Dictionary Methods

Python dictionaries have several built-in methods that help us perform various operations easily.

No.	Method	Description	Syntax	Example	Output / Result
1.	<code>clear()</code>	Removes all items from the dictionary.	<code>dict.clear()</code>	<code>d = {'a':1, 'b':2}</code> <code>d.clear()</code> <code>d</code>	<code>{}</code>
2.	<code>copy()</code>	Returns a shallow copy of the dictionary.	<code>dict.copy()</code>	<code>d = {'a':1, 'b':2}</code> <code>d2 = d.copy()</code> <code>d2</code>	<code>{'a':1, 'b':2}</code>
3.	<code>fromkeys(seq, value)</code>	Returns a new dictionary with keys from seq and value set to value.	<code>dict.fromkeys(seq, value)</code>	<code>keys = ['a', 'b', 'c']</code> <code>d = dict.fromkeys(keys, 0)</code> <code>d</code>	<code>{'a':0, 'b':0, 'c':0}</code>
4.	<code>get(key, default)</code>	Returns the value for key if key is in the dictionary, else returns default.	<code>dict.get(key, default)</code>	<code>d = {'a':1, 'b':2}</code> <code>d.get('a')</code> <code>d.get('x', 'Not Found')</code>	<code>1</code> <code>'Not Found'</code>
5.	<code>items()</code>	Returns a view object containing key:value pairs as tuples.	<code>dict.items()</code>	<code>d = {'a':1, 'b':2}</code> <code>d.items()</code>	<code>dict_items([('a', 1), ('b', 2)])</code>
6.	<code>keys()</code>	Returns a view object containing all the keys.	<code>dict.keys()</code>	<code>d = {'a':1, 'b':2}</code> <code>d.keys()</code>	<code>dict_keys(['a', 'b'])</code>
7.	<code>pop(key, default)</code>	Removes the item with specified key and returns its value. If key not found, returns default.	<code>dict.pop(key, default)</code>	<code>d = {'a':1, 'b':2}</code> <code>d.pop('a')</code> <code>d.pop('x', 'Not Found')</code> <code>d</code>	<code>1</code> <code>'Not Found'</code> <code>{'b':2}</code>
8.	<code>popitem()</code>	Removes and returns the last inserted key:value pair as a tuple.	<code>dict.popitem()</code>	<code>d = {'a':1, 'b':2}</code> <code>d.popitem()</code> <code>d</code>	<code>('b', 2)</code> <code>{'a':1}</code>
9.	<code>update(other_dict)</code>	Updates the dictionary with key:value pairs from another dictionary.	<code>dict.update(other_dict)</code>	<code>d = {'a':1}</code> <code>d.update({'b':2, 'c':3})</code> <code>d</code>	<code>{'a':1, 'b':2, 'c':3}</code>

Example (Combined)

```
d = {'a':1, 'b':2}
d.get('a')      # 1
d.keys()        # dict_keys(['a', 'b'])
d.values()      # dict_values([1, 2])
d.items()       # dict_items([('a', 1), ('b', 2)])
d.update({'c':3}) # {'a':1, 'b':2, 'c':3}
d.pop('b')      # 2
d.popitem()     # ('c', 3)
d.clear()       # {}
```

Points to Remember :

- Methods that return view objects (`keys()`, `values()`, `items()`), reflect changes in the dictionary.
- `get()` and `pop()` with default values are safe to use when key may not exist.
- `update()` adds new key:value pairs or updates existing keys.
- `popitem()` removes the last inserted item (in Python 3.7+ dictionaries maintain insertion order).
- `clear()` makes the dictionary empty.

Unit - IV



Tuples, and Sets : Creating Tuples

1. Introduction to Tuples

- A tuple in Python is an ordered collection of items.
- Tuples are similar to lists, but they are immutable (cannot be changed after creation).
- Tuples are written inside round brackets ()
- Items in a tuple are separated by commas ,

2. Creating Tuples

Tuples can be created in the following ways:

(a) Creating an Empty Tuple

```
t1 = () # Empty tuple
```

Example :

```
t1 = ()
print(t1) # Output → ()
```

(b) Creating a Tuple with Values

```
t2 = (value1, value2, value3, ...)
```

Example :

```
t2 = (10, 20, 30, 40)
print(t2) # Output → (10, 20, 30, 40)
```

Key Points about Tuples

- Tuples are immutable (cannot be changed).
- Tuples maintain insertion order.
- Tuples can contain duplicate values.
- Tuples can store any type of data.

(c) Creating a Tuple with Different Data Types

```
t3 = (value1, value2, value3, ...)
```

Example :

```
t3 = ("Saqib", 17, 5.8, True)
print(t3) # Output → ('Saqib', 17, 5.8, True)
```

(d) Creating a Tuple with One Item

```
t4 = (value, )
```

Comma is important, otherwise it is not considered a tuple.

Example :

```
t4 = (100, )
print(t4) # Output → (100,)
```

Note : t5 = (100) → This is not a tuple, it is an integer.

(e) Creating a Tuple without Parentheses (Using Commas)

```
t5 = 10, 20, 30, 40
```

Parentheses are optional

Example :

```
t5 = 10, 20, 30, 40
print(t5) # Output → (10, 20, 30, 40)
```

Examples Summary

t1 = ()	# Empty tuple
t2 = (10, 20, 30, 40)	# Tuple with multiple values
t3 = ('Saqib', 17, 5.8, True)	# Tuple with different data types
t4 = (100,)	# Tuple with one item
t5 = 10, 20, 30, 40	# Tuple without parentheses

Basic Tuple Operations

* 1. Creation of Tuple

$t = (10, 20, 30, 40)$ # using parentheses
 $t_2 = 5, 'a', 3.2, \text{True}$ # parentheses optional

* 2. Accessing Elements (Indexing)

$t = ('a', 'b', 'c', 'd')$
 $t[0] \rightarrow 'a'$ (first element)
 $t[2] \rightarrow 'c'$ (third element)
 $t[-1] \rightarrow 'd'$ (last element)
 $t[-2] \rightarrow 'c'$ (second last element)

* 3. Slicing

$t = (10, 20, 30, 40, 50, 60)$
 $t[1:4] \rightarrow (20, 30, 40)$
 $t[:3] \rightarrow (10, 20, 30)$
 $t[3:] \rightarrow (40, 50, 60)$
 $t[::2] \rightarrow (10, 30, 50)$

4. Concatenation

$t_1 = (1, 2, 3)$
 $t_2 = ('a', 'b')$
 $t_3 = t_1 + t_2$

$t_3 \rightarrow (1, 2, 3, 'a', 'b')$

5. Repetition

$t = (7, 8, 9)$
 $t_2 = t * 3$

$t_2 \rightarrow (7, 8, 9, 7, 8, 9, 7, 8, 9)$

6. Membership Test

$t = (10, 20, 30, 40)$
 $20 \text{ in } t \rightarrow \text{True}$
 $50 \text{ in } t \rightarrow \text{False}$
 $30 \text{ not in } t \rightarrow \text{False}$

7. Length

$t = (10, 20, 30, 40, 50)$
 $\text{len}(t) \rightarrow 5$

8. Count

$t = (1, 2, 2, 3, 2, 4)$
 $t.\text{count}(2) \rightarrow 3$
 (returns number of occurrences)

9. Index

$t = ('a', 'b', 'c', 'd', 'b')$
 $t.\text{index}('c') \rightarrow 2$
 $t.\text{index}('b') \rightarrow 1$ (first occurrence)

Note :

- Tuples are immutable (cannot be changed).
- Once created, we cannot add, remove or update elements.
- Tuple operations return new tuples.

Indexing and Slicing in Tuples

1. Indexing in Tuples

→ Indexing means accessing a single element of a tuple using its position (index).

Example: `t = ('a', 'b', 'c', 'd', 'e')`

	0	1	2	3	4
<u>Positive Index</u> →	'a'	'b'	'c'	'd'	'e'
<u>Negative Index</u> →	-5	-4	-3	-2	-1

<code>t[0]</code>	→	'a'	(first element)
<code>t[1]</code>	→	'b'	(second element)
<code>t[2]</code>	→	'c'	(third element)
<code>t[4]</code>	→	'e'	(last element)
<code>t[-1]</code>	→	'e'	(last element)
<code>t[-2]</code>	→	'd'	(second last element)
<code>t[:-5]</code>	→	'a'	(first element)

Note: Index must be an integer.

If index is out of range, it gives `IndexError`.

2. Slicing in Tuples

→ Slicing means extracting a part (subtuple) from a tuple.

Syntax: `t[start : stop : step]`

- start → starting index (inclusive)
- stop → ending index (exclusive)
- step → step size (default is 1)

Example: `t = ('a', 'b', 'c', 'd', 'e')`

	0	1	2	3	4
<u>Index</u> →	'a'	'b'	'c'	'd'	'e'
	-5	-4	-3	-2	-1

Expression	Result	Explanation
<code>t[1:4]</code>	('b', 'c', 'd')	from index 1 to 3
<code>t[:3]</code>	('a', 'b', 'c')	from start to index 2
<code>t[2:]</code>	('c', 'd', 'e')	from index 2 to end
<code>t[:]</code>	('a', 'b', 'c', 'd', 'e')	whole tuple copy
<code>t[:2]</code>	('a', 'b')	step of 2
<code>t[1::2]</code>	('b', 'd')	start at 1, step of 2
<code>t[::-1]</code>	('e', 'd', 'c', 'b', 'a')	reverse of tuple
<code>t[4:1:-1]</code>	('e', 'd', 'c')	from index 4 to 2 (backward)
<code>t[:-2]</code>	('a', 'b', 'c')	upto second last element
<code>t[-4:-1]</code>	('b', 'c', 'd')	using negative indices

Note: Slicing always returns a new tuple.

Built-In Functions Used on Tuples

Function	Description	Syntax	Example	Result	Notes
1. <u>len()</u>	Returns the number of elements in the tuple.	<code>len(tuple)</code>	<code>t = (10, 20, 30, 40)</code> <code>len(t)</code>	4	Works on any tuple (empty or non-empty).
2. <u>max()</u>	Returns the largest element in the tuple.	<code>max(tuple)</code>	<code>t = (10, 20, 30, 40)</code> <code>max(t)</code>	40	All elements must be of comparable type.
3. <u>min()</u>	Returns the smallest element in the tuple.	<code>min(tuple)</code>	<code>t = (10, 20, 30, 40)</code> <code>min(t)</code>	10	All elements must be of comparable type.
4. <u>sum()</u>	Returns the sum of all numeric elements in the tuple.	<code>sum(tuple)</code>	<code>t = (1, 2, 3, 4, 5)</code> <code>sum(t)</code>	15	Works only with numbers. Tuple must not be empty.
5. <u>sorted()</u>	Returns a new sorted list from the elements of the tuple.	<code>sorted(tuple)</code>	<code>t = (3, 1, 4, 2)</code> <code>sorted(t)</code>	[1, 2, 3, 4] (list)	Returns a list, not a tuple. Use <code>tuple(sorted(t))</code> to convert back to tuple.
6. <u>any()</u>	Returns True if any element in the tuple is True (or non-zero).	<code>any(tuple)</code>	<code>t = (0, False, 5, "", True)</code> <code>any(t)</code>	True	Checks if at least one element is True / non-zero.
7. <u>all()</u>	Returns True if all elements in the tuple are True (or non-zero).	<code>all(tuple)</code>	<code>t = (1, 2, 3, 4)</code> <code>all(t)</code>	True	Returns False if any element is False / zero.
8. <u>tuple()</u>	Converts an iterable (list, string, etc.) into a tuple.	<code>tuple(iterable)</code>	<code>tuple([1, 2, 3]) → (1, 2, 3)</code> <code>tuple('abc') → ('a', 'b', 'c')</code> <code>tuple(123) → Error</code>	(1, 2, 3) ('a', 'b', 'c') TypeError	If a single number is given, it gives TypeError .

Important Notes :

- Tuples are immutable (cannot be changed).
- Built-in functions do not modify the original tuple.

Example : `t = (5, 2, 8, 1)`

`len(t) = 4` , `max(t) = 8` , `min(t) = 1` , `sum(t) = 16`
`sorted(t) → [1, 2, 5, 8]` , `any(t) → True` , `all(t) → True`

Relation between Tuples and Lists

Tuples and Lists are both sequence data types in Python.

Both are used to store multiple values in a single variable.

Both support indexing, slicing, iteration and membership operations.

Feature	List	Tuple
1. Syntax	[1, 2, 3]	(1, 2, 3)
2. Mutability	Mutable (can be changed)	Immutable (cannot be changed)
3. Can modify elements?	Yes	No
4. Methods	More methods available (e.g., append(), extend(), insert(), remove(), sort() etc.)	Fewer methods available (count(), index() only)
5. Speed	Slightly slower	Slightly faster
6. Use	When data may change	When data should remain fixed (or constant)
7. Example	marks = [80, 90, 85]	marks = (80, 90, 85)

Similarities :

- Both are sequence data types.
- Both support indexing and slicing.
- Both can store different data types.
- Both support iteration (using loops).
- Both support membership operators: in and not in.

Differences (Key Point) :

- Lists are mutable, tuples are immutable.
- Lists have more built-in methods, tuples have fewer.
- Tuples are faster than lists.
- Use list for changeable data, tuple for fixed data.

Example :

```
# List
list1 = [10, 20, 30]
list1[0] = 100 # Allowed
print(list1) # [100, 20, 30]

# Tuple
tuple1 = (10, 20, 30)
# tuple1[0] = 100 # Error
print(tuple1) # (10, 20, 30)
```

In Short :

Lists and tuples are similar because both store ordered collections of elements. But the main difference is that lists can be changed (mutable) while tuples cannot be changed (immutable) after creation.

Relation between Tuples and Dictionaries

- Tuples and Dictionaries are built-in data types in Python.
- Both are used to store multiple values, but they differ in structure, usage and properties.

Feature	Tuple	Dictionary
1. Definition	Tuple is an ordered collection of elements.	Dictionary is a collection of key-value pairs.
2. Syntax	<code>t = (1, 2, 3)</code>	<code>d = {'a': 1, 'b': 2}</code>
3. Order	Maintains order.	From Python 3.7+, maintains insertion order.
4. Access	Accessed by index (position).	Accessed by key.
5. Mutability	Immutable (cannot be changed after creation).	Mutable (can be changed after creation).
6. Elements	Stores values (any data type).	Stores key-value pairs.
7. Duplicates	Allows duplicate values.	Keys must be unique. Values can be duplicate.
8. Use	Used when we want to store multiple items as a single unit (fixed data).	Used when we want to store labeled data (key-value form).
9. Example	<code>t = ('a', 10, 3.5)</code>	<code>d = {'name': 'Ali', 'age': 20}</code>

Similarities :

- Both are used to store multiple values in a single variable.
- Both can store mixed data types (int, float, string, etc.).
- Both support iteration using loops.
- Both are built-in data types in Python.

Examples :

Tuple

```
t = (10, 20, 30)
```

```
t[0] → 10
```

```
t[1] → 20
```

```
t[2] → 30
```

```
# t[0] = 100 → Error
(Because tuple is immutable)
```

Dictionary

```
d = {'a': 10, 'b': 20}
```

```
d['a'] → 10
```

```
d['b'] → 20
```

```
d['a'] = 100 → Allowed
```

```
d['c'] = 30 → Allowed
```

Key Point :

Use Tuple when you need an ordered, fixed collection of items.

Use Dictionary when you need data in key-value pairs for easy access.

Tuple Method

Tuples are immutable, i.e., once created, we cannot change their elements. But Python provides some useful methods.

1. `Count()` – Returns the number of times a specified element occurs in tuple.

Syntax : `tuple.count(element)`

Example :

```
t = (1, 2, 3, 2, 4, 2)
t.count(2) → 3
```

2. `index()` – Returns the index of the first occurrence of a specified element.

Syntax : `tuple.index(element)`

Example :

```
t = (10, 20, 30, 40, 20)
t.index(20) → 1
```

3. `len()` – Returns the number of elements in the tuple.

Syntax : `len(tuple)`

Example :

```
t = (5, 10, 15, 20)
len(t) → 4
```

4. `min()` – Returns the smallest element in the tuple.

Syntax : `min(tuple)`

Example :

```
t = (8, 3, 6, 2, 10)
min(t) → 2
```

5. `max()` – Returns the largest element in the tuple.

Syntax : `max(tuple)`

Example :

```
t = (8, 3, 6, 2, 10)
max(t) → 10
```

6. `tuple()` – Converts another sequence (list, string, etc.) into tuple.

Syntax : `tuple(sequence)`

Example :

```
lst = [1, 2, 3, 4]
t = tuple(lst) → (1, 2, 3, 4)
```

Note : Tuples being immutable do not have methods like `append()`, `insert()`, `remove()`, etc.

Syntax : `zip(iterable1, iterable2, ...)`

Example :

```
# Converting the zip object to a list to display the result
result = list(z)
print(result)
```

Note :

- `zip()` stops when the shortest iterable is exhausted.
- The result of `zip()` is an iterator (zip object).
Convert it using `list()`, `tuple()`, etc., to see the output.
- It is commonly used in loops.

How it works (element-wise pairing)

$[('Alice', 85), ('Bob', 90), ('Charlie', 95)]$

If iterables are of unequal length :

It stops at the shortest one.

Example :

```
a = [1, 2, 3, 4]
b = ['a', 'b']
list(zip(a, b)) → [(1, 'a'), (2, 'b')]
```

Sets in Python

A set is an unordered collection of unique elements.

- Elements in a set must be immutable (int, float, str, tuple, etc.).
- Sets do not allow duplicate values.
- Sets are mutable (we can add or remove elements).

1. Creating Sets

Using {} (curly braces)

`s = {1, 2, 3, 4}`

Using set() constructor

`s = set([1, 2, 3])`

Empty set

`s = set()` # not `s = {}` (which is a dict)

2. Properties of Sets

- Unordered - elements have no fixed order.
- Unique - duplicate elements are not allowed.
- Mutable - we can add or remove elements.
- Heterogeneous - can store different data types.

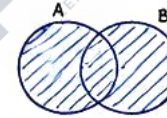
3. Basic Operations

- `add(x)` → Adds an element `x` to the set
- `remove(x)` → Removes `x`; raises `KeyError` if not present
- `discard(x)` → Removes `x`; does not raise error if not present
- `pop()` → Removes and returns an arbitrary element
- `clear()` → Removes all elements from the set
- `len(s)` → Returns number of elements in set
- `x in s` → Returns True if `x` is present in set
- `x not in s` → Returns True if `x` is not present in set

4. Set Operations

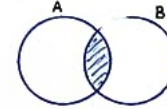
- (1) Union (|) - elements in A or B or both

$A \cup B$



- (2) Intersection (&) - common elements

$A \cap B$



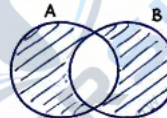
- (3) Difference (-) - elements in A but not in B

$A - B$



- (4) Symmetric Difference (^) - elements in A or B but not in both

$A \Delta B$



- (5) Subset / Superset

$A \leq B$ → A is subset of B

$A \geq B$ → A is superset of B

$A < B$ → proper subset

$A > B$ → proper superset

Example :

$A = \{1, 2, 3\}$

$B = \{3, 4, 5\}$

$A \cup B = \{1, 2, 3, 4, 5\}$

Example :

$A \cap B = \{3\}$

Example :

$A - B = \{1, 2\}$

Example :

$A \Delta B = \{1, 2, 4, 5\}$

Example :

$A = \{1, 2\}$, $B = \{1, 2, 3, 4\}$

$A \leq B$ → True

$A < B$ → True

$B > A$ → True

5. Some Set Methods

- `s.add(x)` → Adds `x` to set
- `s.remove(x)` → Removes `x`; raises error if not present
- `s.discard(x)` → Removes `x`; no error if not present
- `s.pop()` → Removes and returns an arbitrary element
- `s.clear()` → Removes all elements

Note :

- {} is used for set with elements.
- set() is used to create empty set.

Set Methods in Python

Python sets come with a number of built-in methods that allow us to add, remove, and perform operations on sets.

1. `add(x)` - Adds an element `x` to the set.

Example: `s = {1, 2, 3}`
`s.add(4)`
`s` → {1, 2, 3, 4}

2. `update(iterable)` - Adds all elements from the iterable to the set.

Example: `s = {1, 2}`
`s.update([2, 3, 4])`
`s` → {1, 2, 3, 4}

3. `remove(x)` - Removes element `x` from the set.
 Raises `KeyError` if `x` is not present.

Example: `s = {1, 2, 3}`
`s.remove(2)`
`s` → {1, 3}

4. `discard(x)` - Removes element `x` from the set.
 Does not raise error if `x` is not present.

Example: `s = {1, 2, 3}`
`s.discard(4)`
`s` → {1, 2, 3} (no error)

5. `pop()` - Removes and returns an arbitrary element from the set.

Example: `s = {1, 2, 3}`
`x = s.pop()` # `x` can be 1 or 2 or 3
`s` → remaining elements

6. `clear()` - Removes all elements from the set.

Example: `s = {1, 2, 3}`
`s.clear()`
`s` → `set()` (empty set)

Note: All these methods modify the set in-place unless specified otherwise (union, intersection, difference, symmetric_difference return new sets).

7. `union(other)` - Returns a new set with elements from both sets.

(or use `|` operator)

Example: `A = {1, 2, 3}` `B = {3, 4, 5}`
`A.union(B)` → {1, 2, 3, 4, 5}
`A | B` → {1, 2, 3, 4, 5}

8. `intersection(other)` - Returns a new set with elements common to both sets.

(or use `&` operator)

Example: `A = {1, 2, 3}` `B = {3, 4, 5}`
`A.intersection(B)` → {3}
`A & B` → {3}

9. `difference(other)` - Returns a new set with elements in the first set but not in the other.

(or use `-` operator)

Example: `A = {1, 2, 3}` `B = {3, 4, 5}`
`A.difference(B)` → {1, 2}
`A - B` → {1, 2}

10. `symmetric_difference(other)` - Returns a new set with elements in either set, but not in both.

(or use `^` operator)

Example: `A = {1, 2, 3}` `B = {3, 4, 5}`
`A.symmetric_difference(B)` → {1, 2, 4, 5}
`A ^ B` → {1, 2, 4, 5}

11. `issubset(other)` - Returns True if set is subset of other set.

(or use `<=` operator)

Example: `A = {1, 2}` `B = {1, 2, 3}`
`A.issubset(B)` → True
`A <= B` → True

12. `issuperset(other)` - Returns True if set is superset of other set.

(or use `>=` operator)

Example: `A = {1, 2, 3}` `B = {1, 2}`
`A.issuperset(B)` → True
`A >= B` → True

Traversing of Sets

Traversing means accessing each element of a set one by one. In Python, we can traverse a set using a loop.

1. Using for loop

- The for loop is the most common way to traverse a set.
- It visits each element in the set.

Example :

```
s = {10, 20, 30, 40, 50}
print("Elements of set are:")
for x in s:
    print(x)
```

Output (order may vary)

```
Elements of set are:
40
10
50
20
30
```

Note :

- Sets are unordered, so the elements may not appear in the same order every time.

2. Using while loop and iterator

- We can also traverse a set by first creating an iterator using `iter()` and then using `next()`.
- This method gives more control over iteration.

Example :

```
s = {'a', 'b', 'c', 'd'}
it = iter(s)
print("Elements of set are:")
while True:
    try:
        x = next(it)
        print(x)
    except StopIteration:
        break
```

Output (order may vary)

```
Elements of set are:
c
a
b
d
```

Important Points

- We can traverse a set using for loop (simple and easy).
- We can also use iterator with while loop.
- Since sets are unordered, the traversal order is not fixed.
- Duplicate elements are not allowed in a set.

Frozenset

A frozenset is an immutable version of set in Python.

It cannot be changed once created.

Being immutable, it can be used as a key in dictionaries or as an element in another set.

Key Points

- frozenset is immutable (unchangeable).
- It supports set operations like union, intersection, etc.
- It can be used as a dictionary key.
- It can be an element of another set.

1. Creating FrozenSet

```
fs = frozenset(iterable)
# iterable can be list, tuple, set, string, etc.
```

Examples :

- `fs1 = frozenset([1, 2, 3, 4])` → `frozenset({1, 2, 3, 4})`
- `fs2 = frozenset("hello")` # duplicate removed → `frozenset({'h', 'e', 'l', 'o'})`
- `fs3 = frozenset((10, 20, 30))` → `frozenset({10, 20, 30})`

2. Properties

- Immutable → cannot add, remove or change elements.
- Unordered → elements are not stored in any fixed order.
- Unique elements → duplicate values are not allowed.

Note :

Since frozenset is immutable, methods like `add()`, `remove()`, `clear()`, `pop()` are not available.

5. Difference between set and frozenset

Feature	set	frozenset
Mutability	Mutable (can be changed)	Immutable (cannot be changed)
Methods	<code>add()</code> , <code>remove()</code> , <code>clear()</code> , <code>pop()</code> available	<code>add()</code> , <code>remove()</code> , <code>clear()</code> , <code>pop()</code> not available
Usage	General purpose	Can be used as key in dict or as element in another set
Syntax	<code>{1, 2, 3}</code> or <code>set()</code>	<code>frozenset([1, 2, 3])</code>

3. Set Operations (supported)

Frozenset supports most of the set operations.

Operation	Example	Output
Union ()	<code>fs1 fs2</code>	<code>frozenset({1, 2, 3, 4, 5})</code>
Intersection (&)	<code>fs1 & fs2</code>	<code>frozenset({3, 4})</code>
Difference (-)	<code>fs1 - fs2</code>	<code>frozenset({1, 2})</code>
Symmetric Difference (^)	<code>fs1 ^ fs2</code>	<code>frozenset({1, 2, 5})</code>
Subset (<=)	<code>fs1 <= fs2</code>	True / False
Superset (>=)	<code>fs1 >= fs2</code>	True / False

Example :

```
fs1 = frozenset([1, 2, 3, 4])    fs2 = frozenset([3, 4, 5])
```

```
fs1 | fs2    → frozenset({1, 2, 3, 4, 5})
fs1 & fs2    → frozenset({3, 4})
fs1 - fs2    → frozenset({1, 2})
fs1 ^ fs2    → frozenset({1, 2, 5})
```

4. Using FrozenSet

Since it is immutable, it can be used as :

- Dictionary key
- Element of another set

Example (Dictionary Key)

```
d = {frozenset({1, 2}) : 'A',
      frozenset({3, 4}) : 'B'}
print(d[frozenset({1, 2})])
# Output: A
```

Example (Element of a set)

```
s = {frozenset({1, 2}),
      frozenset({3, 4})}
print(s)
# Output: {frozenset({1, 2}),
           frozenset({3, 4})}
```

In short :

`frozenset = set`
but frozen (unchangeable) !